

UNIVERSIDAD NACIONAL DE AVELLANEDA



MATERIA: Arquitectura de Sistemas de Elaboración de Datos II

Trabajo de Laboratorio N°2

Integrantes:

Apellido y Nombres

E-mails

Fusaro Joel

joelfusaro1@gmail.com

Pietrella Emma

emma.pietrella@gmail.com

López Camila Marisol

camilamarisol2005@gmail.com

Ponce Brisa

brisa_ponce3@hotmail.com

Profesor:

Ing.Matías Loiseau / e-mail: mloiseau@undav.edu

2do Cuatrimestre 2026

Resumen

El presente trabajo consiste en el diseño e implementación de una estación meteorológica basada en ESP32 y programada mediante Arduino. El sistema permitirá medir temperatura, humedad y presión atmosférica utilizando los sensores DS18B20, DHT22 y BMP280. Las mediciones serán almacenadas y visualizadas mediante una interfaz web en la red local y en un servicio cloud. Además, se implementará un sistema de “offline resilience” que permitirá continuar registrando datos ante una pérdida de conexión a Internet y sincronizarlos automáticamente cuando la conectividad sea restablecida.

Índice

Objetivo.....	3
Introducción.....	4
Marco teórico.....	5
Componentes y sus conexiones.....	5
Microcontrolador ESP32.....	5
Sensor DHT22.....	6
Sensor DS18B20.....	7
Sensor BMP280.....	8
Comunicación I ² C.....	8
Comunicación Wi-Fi e IoT.....	10
Almacenamiento local y Cloud.....	10
Offline Resilience.....	10
Desarrollo.....	11
Intervalo de medición.....	12
Servidor web local y visualización mediante LAN (Local Area Network).....	13
Comunicación con ThingSpeak.....	14
Almacenamiento local mediante LittleFS (memoria flash interna).....	14
Validación de datos.....	17
Recuperación de la conectividad y sincronización.....	18
Resultados.....	19
Conclusiones.....	24
Mejoras a desarrollar.....	25
Apéndice.....	26
Bibliografía.....	27

Objetivo

Diseñar una estación meteorológica que mida variables ambientales, almacene y visualice dichos datos localmente y en la nube. Por último el sistema debe ser *“offline resilience”*. Además, el sistema deberá realizar las siguientes acciones: medir temperatura, humedad y presión. Visualizar las variables en la web, tanto local como cloud.

Introducción

Una estación meteorológica es un sistema destinado a la adquisición y registro de diferentes variables ambientales, permitiendo analizar su comportamiento a lo largo del tiempo. En el presente trabajo se desarrollará una estación basada en un microcontrolador ESP32, programado mediante la plataforma Arduino.

El sistema utilizará un sensor DHT22 para medir la humedad relativa, un DS18B20 para la temperatura y un BMP280 para la presión atmosférica. Las mediciones obtenidas serán procesadas por el microcontrolador y almacenadas para su posterior consulta y análisis.

El proyecto también aborda conceptos relacionados con el Internet de las Cosas (IoT), debido a que el sistema contará con conectividad Wi-Fi para transmitir los datos hacia un servicio en la nube. Paralelamente, se implementará una interfaz web accesible desde la red local para visualizar las mediciones, preferentemente mediante gráficos en función del tiempo.

Finalmente, se analizará el concepto de *“offline resilience”*, implementando un mecanismo que permita continuar registrando las mediciones cuando se pierda la conexión a Internet. Una vez restablecida la conectividad, los datos almacenados localmente serán sincronizados con la plataforma cloud, evitando la pérdida de información.

Marco teórico

El desarrollo del presente proyecto requiere la integración de múltiples dominios de la electrónica y la programación. Para comprender el funcionamiento integral del sistema, es imperativo definir los principios físicos y los conceptos teóricos que rigen el comportamiento de los componentes de hardware involucrados.

Componentes y sus conexiones

Microcontrolador ESP32

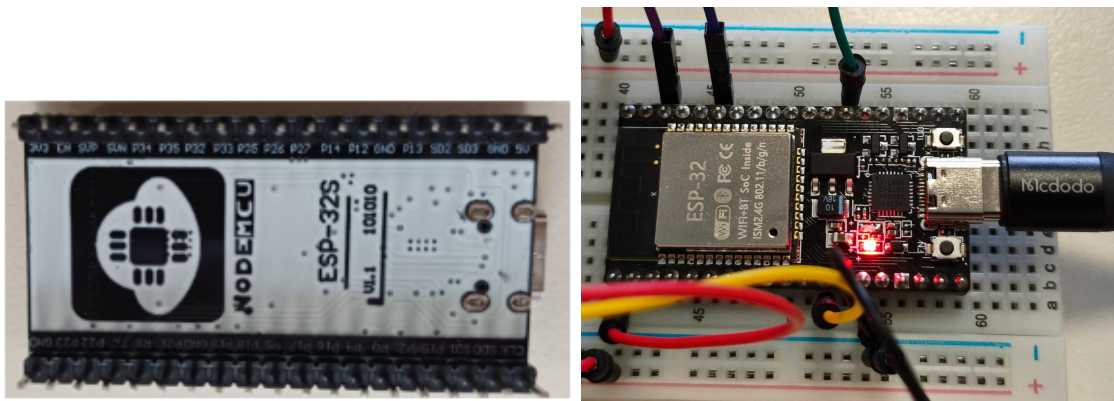


Figura 1 & 2. Microcontrolador ESP32

El ESP32 es un microcontrolador ampliamente utilizado en aplicaciones de Internet de las Cosas (IoT). Además de sus capacidades de procesamiento y sus múltiples entradas y salidas digitales, incorpora conectividad Wi-Fi, lo que permite establecer comunicación con redes locales y servicios disponibles en Internet.

En este proyecto, el ESP32 será el elemento central de la estación meteorológica. Se encargará de adquirir las mediciones de los sensores, procesarlas, almacenarlas y transmitir las hacia la red local y la plataforma cloud.

Sensor DHT22

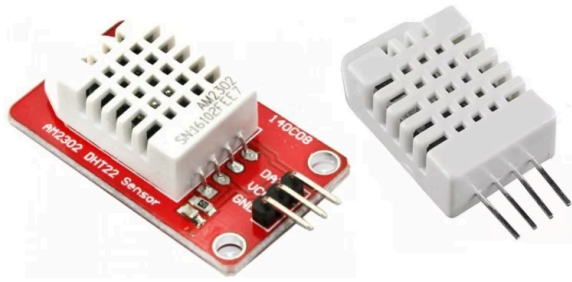


Figura 3. Sensor digital DHT22

El DHT22 es un sensor digital utilizado para medir la humedad relativa y la temperatura del ambiente. En este proyecto se utilizará principalmente para obtener el valor de humedad.

La comunicación con el microcontrolador se realiza mediante un protocolo de comunicación digital de un solo hilo (single bus). A diferencia de los buses compartidos, este esquema opera mediante una línea dedicada exclusiva (no permite compartir el pin con ningún otro sensor) y se basa en una secuencia específica de pulsos de tiempo para sincronizar la adquisición de los datos. Su integración en el firmware no requiere configurar periféricos internos del microcontrolador, ya que la temporización y decodificación de los datos son gestionadas directamente por la librería del fabricante.

El sensor requiere alimentación, conexión a masa y una línea de datos conectada a uno de los GPIO del ESP32.

Sensor DS18B20



Figura 4. Sensor digital DHT22

El DS18B20 es un sensor digital de temperatura que utiliza el protocolo de comunicación 1-Wire. Este protocolo permite realizar una comunicación bidireccional utilizando una única línea de datos, además de las conexiones de alimentación y masa.

Una característica importante del DS18B20 es que proporciona directamente la temperatura digitalizada, evitando la necesidad de utilizar un conversor analógico-digital externo. Para garantizar una correcta comunicación, normalmente se utiliza una resistencia de pull-up en la línea de datos.

Sensor BMP280

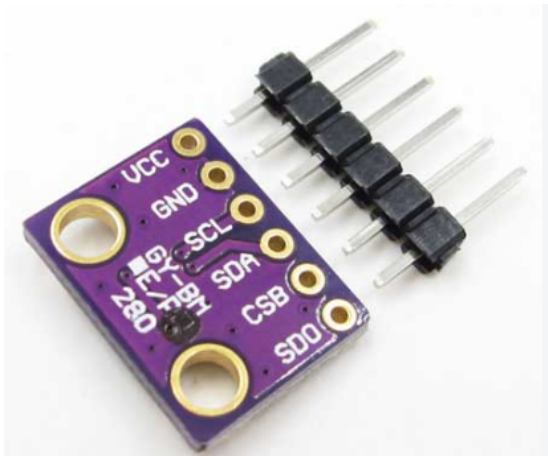


Figura 5. Sensor digital BMP280

El BMP280 es un sensor digital utilizado para medir la presión atmosférica. Puede comunicarse mediante los protocolos I²C o SPI. En este proyecto se utilizará preferentemente I²C como protocolo para establecer la comunicación entre el ESP32 y el BMP280, debido a que permite una conexión sencilla mediante las líneas SDA y SCL.

La presión atmosférica obtenida se expresa en hectopascales (hPa), unidad habitual en aplicaciones meteorológicas.

Comunicación I²C

I²C es un protocolo de comunicación serial que permite conectar diferentes dispositivos mediante dos líneas principales: SDA, utilizada para el intercambio de datos, y SCL, utilizada como señal de reloj.

El BMP280 es capaz de comunicarse tanto por I²C como por SPI; el protocolo que efectivamente use no depende de una configuración por software, sino del nivel de tensión que se le aplica al pin CSB (Chip Select Bar) en el hardware. En este proyecto, se conectó CSB directamente al riel positivo (3.3V) de la protoboard, fijando el modo I²C de forma permanente para todas las lecturas.

En el modo I²C, el pin SDO (Serial Data Out) no participa activamente en la transferencia de datos, sino que se utiliza para definir la dirección física que

identificará al sensor. Esta dirección, expresada en formato hexadecimal, funciona como un identificador único en el bus de datos, garantizando que el microcontrolador establezca un canal de comunicación unívoco con el BMP280. En este caso, se optó por dejar el pin conectado a GND, lo cual representa la dirección 0x76.

Aprovechando el soporte nativo del ESP32 para este protocolo, se optó por utilizar los dos pines asignados por defecto para el bus I²C: el GPIO21 para la línea SDA (datos) y el GPIO22 para la línea SCL (reloj). De esta manera se facilita la implementación en el firmware, al reducir la inicialización del bus en el código simplemente a la línea `Wire.begin()`, sin necesidad de indicar manualmente qué pines cumplen cada función. Del mismo modo, esta configuración por defecto simplifica el uso de las librerías estándar del sensor, ya que la mayoría están diseñadas para asumir estas conexiones predeterminadas, evitando la necesidad de realizar configuraciones adicionales para la comunicación.

Para ese proyecto se optó por I²C debido a las siguientes razones:

- Requiere una menor cantidad de líneas de datos, solo necesita dos (SDA y SCL), a diferencia del SPI que requiere como mínimo cuatro (MOSI, MISO, SCK y una línea de Chip Select por cada dispositivo conectado). Logrando de esta manera simplificar la interconexiones del sistema y optimizar el uso de pines del microcontrolador.
- Asimismo, la reducción de la cantidad de líneas de datos, disminuye notablemente la probabilidad de errores en el hardware. Se reducen los riesgos de conexiones flojas o falsos contactos, se minimiza el ruido magnético e interferencias y se facilita la depuración de errores en el circuito.
- Al contar con un soporte nativo por hardware para este protocolo, se simplifica su implementación en el firmware, sin necesidad de configurar manualmente líneas de selección adicionales como exigiría SPI.

Comunicación Wi-Fi e IoT

La conectividad Wi-Fi permite que el ESP32 se comuniquen con otros dispositivos dentro de una red LAN y, mediante una conexión a Internet, con servidores y plataformas cloud.

Esta característica permite incorporar el proyecto dentro del concepto de Internet de las Cosas (IoT), donde un dispositivo físico puede adquirir información del entorno y transmitirla para su almacenamiento, análisis y visualización remota.

Almacenamiento local y Cloud

El almacenamiento local permite conservar las mediciones directamente en el sistema. Por otro lado, el almacenamiento cloud permite disponer de los datos desde Internet y facilita su representación mediante gráficos.

La combinación de ambos mecanismos permite implementar una arquitectura más robusta, ya que los datos pueden conservarse localmente aunque temporalmente no exista conexión con el servidor remoto.

En este caso utilizaremos la plataforma en la nube **ThingSpeak**, encargada de hacer un análisis para el Internet de las Cosas (IoT) que permite recolectar, almacenar, visualizar y analizar flujos de datos en tiempo real de cada sensor.

Offline Resilience

El concepto de “offline resilience” hace referencia a la capacidad de un sistema para continuar funcionando ante una interrupción de la conectividad.

En este proyecto, cuando el ESP32 no pueda comunicarse con Internet, las mediciones continuarán almacenándose localmente. Una vez restablecida la conexión, los datos pendientes serán enviados al servicio cloud y sincronizados con la información previamente almacenada.

Esta estrategia permite evitar la pérdida de mediciones debido a interrupciones temporales de Internet.

Desarrollo

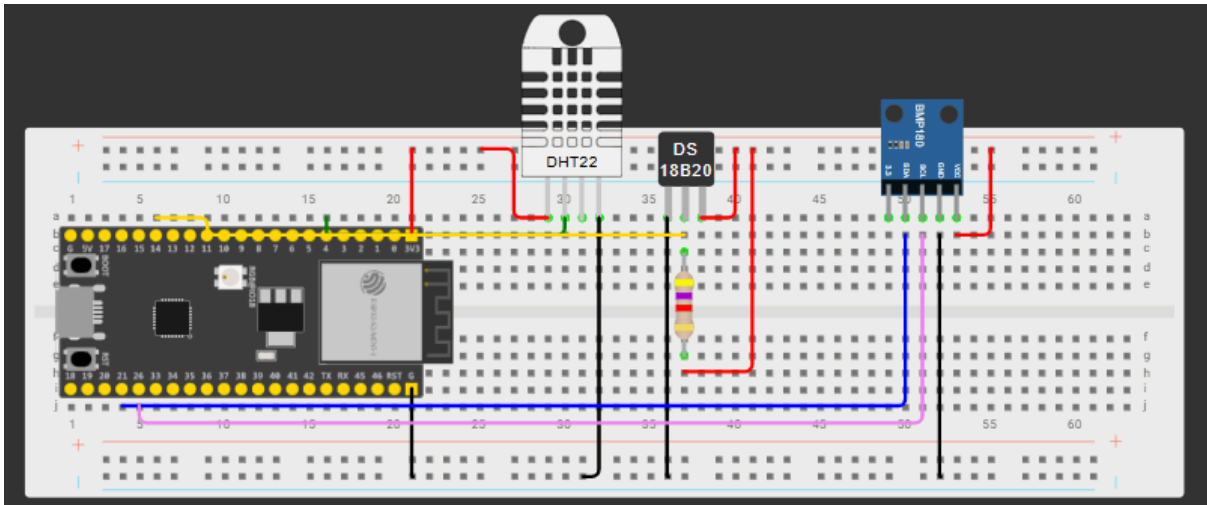


Figura 6 simulación del circuito con sus componentes

El proyecto desarrollado consiste en una estación meteorológica basada en un microcontrolador ESP32, cuya función principal es adquirir, procesar, almacenar y transmitir información correspondiente a diferentes variables ambientales. Para realizar las mediciones se utilizaron tres sensores: DHT22, DS18B20 y BMP280, destinados a obtener información de humedad, temperatura y presión atmosférica, respectivamente.

En el esquema ilustrativo (Figura 6) se empleó un sensor BMP180 en lugar del BMP280 utilizado en el circuito físico, ya que el simulador no dispone de este último. Ambos sensores pertenecen a la misma familia (Bosch), miden presión barométrica y comparten las líneas principales del bus I²C (VCC, GND, SDA y SCL). La diferencia radica en que el BMP280 ofrece mayor precisión y menor consumo, además de contar con 2 pines adicionales: CSB, conectado a 3.3V para seleccionar el modo de comunicación I²C, y SDO, conectado a tierra (GND) para fijar la dirección I²C en 0x76.

Los tres sensores fueron montados sobre una protoboard y conectados al ESP32. La alimentación de los sensores se realizó utilizando la tensión de 3,3 V

proporcionada por el microcontrolador, junto con las correspondientes conexiones a GND y a los pines de comunicación necesarios para cada dispositivo.

El funcionamiento completo del sistema se encuentra controlado mediante un programa desarrollado en el entorno Arduino IDE. El software se encarga de inicializar los sensores, realizar las mediciones en intervalos determinados, procesar los valores obtenidos, establecer la comunicación Wi-Fi y administrar el almacenamiento y envío de los datos.

Una característica fundamental de la implementación es la capacidad de funcionamiento ante una interrupción de Internet, también conocido como “*offline resilience*”. En este sistema, dicha propiedad se logra mediante el almacenamiento local de las mediciones en un archivo gestionado por el sistema de archivos LittleFS, lo que garantiza la persistencia y posterior sincronización de los datos sin pérdida de información ante eventuales caídas de red.

La estructura general del programa puede dividirse conceptualmente en las siguientes fases:

Fase 1: Configuración inicial (setup)

INICIO → Inicialización de librerías → Configuración de sensores → Inicialización de LittleFS → Conexión a Wi-Fi y sincronización horaria (NTP)

Fase 2: Bucle principal (loop)

Espera del intervalo de medición → Lectura y validación de cada una de las variables meteorológicas → Almacenamiento local en LittleFS → Gestión de sincronización remota (Thingspeak / Servidor web) → Retorno al inicio del bucle

Intervalo de medición

Una de las condiciones establecidas para el funcionamiento de la estación meteorológica fue realizar una medición cada dos minutos.

Por lo tanto, el sistema ejecuta periódicamente el proceso de adquisición de datos. En cada ciclo se consulta el estado de los tres sensores y se obtiene un nuevo conjunto de valores.

Servidor web local y visualización mediante LAN (Local Area Network)

El primer muestreo además del ThinkSpeak, el sistema implementa una interfaz web local para consultar las mediciones desde dispositivos conectados a la misma red LAN que el ESP32.

La interfaz está desarrollada directamente dentro del código Arduino mediante una página HTML almacenada en una constante denominada PAGINA_HTML. La utilización de PROGMEM permite almacenar esta información en la memoria destinada al programa del ESP32, evitando ocupar innecesariamente la memoria RAM.

La página web contiene tres áreas principales de visualización:

- Temperatura obtenida mediante el DS18B20.
- Humedad obtenida mediante el DHT22.
- Presión atmosférica obtenida mediante el BMP280.

La estructura visual se basa en elementos HTML `<canvas>`, sobre los cuales JavaScript dibuja los gráficos correspondientes.

Una característica importante de esta implementación es que la página no utiliza dependencias externas. El HTML, CSS y JavaScript necesarios para generar la interfaz están incorporados directamente en el programa del ESP32.

Por este motivo, la visualización local puede continuar funcionando aunque el dispositivo no tenga acceso a Internet, siempre que el usuario pueda comunicarse con el ESP32 mediante la red local.

Comunicación con ThingSpeak

Para el almacenamiento y visualización de los datos en la nube se utilizó la plataforma **ThingSpeak**, orientada al desarrollo de aplicaciones relacionadas con IoT.

El ESP32 utiliza su conexión Wi-Fi para comunicarse con ThingSpeak y enviar los valores obtenidos de los sensores.

Para realizar la comunicación se utiliza una **API Key**, previamente generada y configurada en la plataforma. Esta clave permite identificar y autorizar al dispositivo para realizar el envío de información al canal correspondiente.

Cada dos minutos, cuando existe conectividad, el ESP32 obtiene las mediciones y las envía a los campos correspondientes del canal configurado en ThingSpeak.

La utilización de ThingSpeak permite visualizar las mediciones de manera gráfica, haciendo posible analizar la evolución de cada variable en función del tiempo.

Almacenamiento local mediante LittleFS (memoria flash interna)

Uno de los aspectos principales del proyecto es evitar la pérdida de información cuando el sistema no dispone de conexión a Internet. Para ello se utiliza **LittleFS (Little File System)**, un sistema de archivos que permite utilizar parte de la memoria flash interna del ESP32 para almacenar información.

Se optó por la memoria flash del ESP32 en lugar de su memoria RAM para almacenar datos, ya que la RAM es una memoria volátil: su contenido se pierde ante un corte de energía o un reinicio del microcontrolador. Dado que el objetivo del

sistema es precisamente resguardar los datos durante una interrupción, sea de Internet o incluso de la propia alimentación eléctrica, utilizar la RAM habría dejado la información vulnerable ante ese mismo tipo de eventos que se busca tolerar. La memoria flash, en cambio, es no volátil, por lo que los datos permanecen almacenados incluso si el ESP32 se apaga o se reinicia de forma inesperada.

Cada lectura válida se guarda siempre en el archivo local `/data.csv`, independientemente de si hay conexión a internet en ese momento. Por separado, el sistema intenta sincronizar esos datos con ThingSpeak, si el envío es exitoso, se actualiza un puntero (`/sync_state.txt`) que indica hasta qué momento ya se sincronizó. En cambio si el envío falla, el dato ya se encuentra almacenado localmente y el sistema simplemente reintentará enviarlo en el próximo ciclo, sin haberlo perdido. Este esquema, al guardar todo localmente sin condicionarlo al resultado del envío, permite además que la página web local pueda mostrar el historial completo de mediciones en todo momento, tenga o no acceso a internet.

El proceso se representa de la siguiente manera:

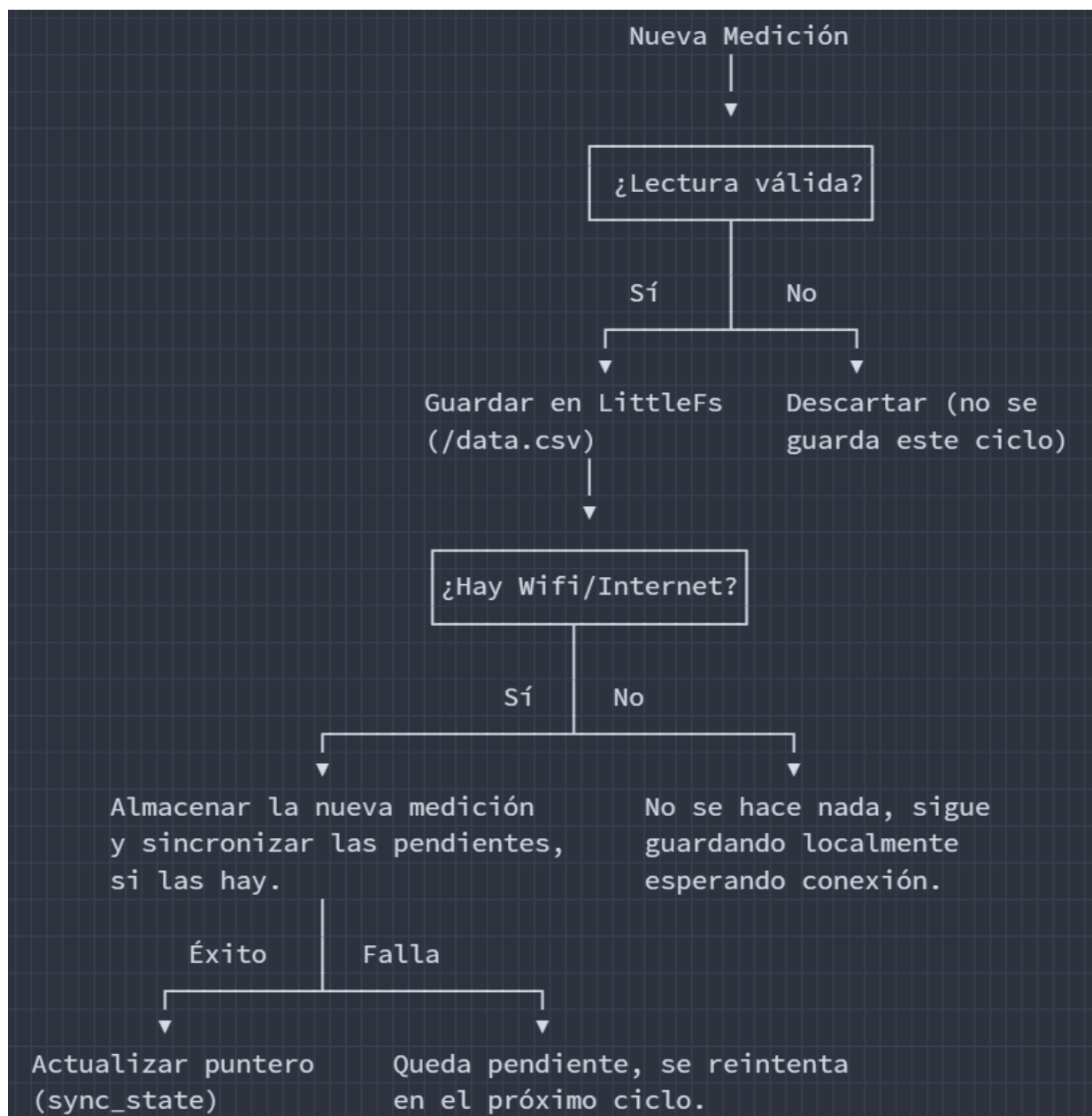


Figura 7 Diagrama de flujo del programa ante una medición de datos

Este mecanismo garantiza que ninguna lectura válida se pierda: el almacenamiento local ocurre siempre, mientras que el envío a la nube se realiza únicamente cuando hay conexión disponible, sin bloquear ni condicionar el guardado de los datos.

Por otro lado, dado que el archivo `/data.csv` almacena de forma acumulativa todas las lecturas válidas, es necesario limitar su crecimiento para evitar que eventualmente ocupe todo el espacio disponible en la memoria flash. Por este motivo, cada vez que se guarda una nueva medición, el sistema verifica el tamaño

actual del archivo, si este supera un límite máximo definido (100 KB en este proyecto), se ejecuta un recorte: se leen todas las líneas almacenadas y se reescribe el archivo conservando únicamente la mitad más reciente, descartando las lecturas más antiguas. Este mecanismo mantiene el archivo dentro de un tamaño acotado de forma indefinida, sin comprometer la sincronización de datos pendientes, ya que dicho proceso se basa en el timestamp de cada registro y no en su posición dentro del archivo.

Nota: El Timestamp (Unix) es el formato en el que el microcontrolador captura y almacena los instantes de tiempo (segundos transcurridos desde la “Época Unix”, 1º de enero de 1970), optimizando el procesamiento interno y el uso de la memoria.

Validación de datos

Como anteriormente nombramos, antes de almacenar cualquier lectura de variables primero el sistema evalúa su validez, esto lo hace a través de la variable `r.valid`. Es importante aclarar que este control no se aplica de la misma manera a los tres sensores. Para el DHT22, se verifica que el valor de humedad devuelto no sea un número inválido (NaN, Not a Number), lo cual ocurre típicamente cuando la comunicación con el sensor falla momentáneamente. Para el DS18B20, se verifica que el sensor no devuelva el valor especial `device_disconnected_c`, que la librería retorna cuando no logra detectar ningún dispositivo respondiendo en el bus de datos. El BMP280, en cambio, no cuenta con esta misma verificación en la implementación actual, ya que al comunicarse por I²C con confirmación de lectura, sus fallos de comunicación son menos frecuentes que en los protocolos de un solo cable utilizados por los otros dos sensores.

Si cualquiera de las dos condiciones controladas (humedad o temperatura) falla, la lectura completa se descarta y no se guarda en `/data.csv` para ese ciclo. Este mecanismo evita que se almacenen datos erróneos o incompletos en la memoria flash, y por extensión, evita que dichos datos incorrectos se propaguen a la plataforma cloud o se muestren en los gráficos de la página web local. Al mismo tiempo, contribuye a un uso más eficiente de los recursos del sistema: no tiene

sentido consumir espacio de almacenamiento ni ancho de banda de sincronización en información que no representa una medición real y confiable del ambiente.

Recuperación de la conectividad y sincronización

A diferencia de un mecanismo que detecta explícitamente "el momento en que vuelve la conexión", el sistema no necesita un estado especial para esto: como se describió en el diagrama de flujo, cada ciclo de muestreo intenta sincronizar lo pendiente si hay WiFi disponible. Esto significa que apenas la conectividad se restablece, el próximo ciclo automáticamente retoma el envío, sin lógica adicional para "detectar la recuperación" como un evento distinto.

El criterio que determina qué se considera pendiente es el puntero guardado en `/sync_state.txt`: toda lectura con timestamp posterior al último dato sincronizado exitosamente se envía a ThingSpeak, respetando el orden cronológico en que fue registrada. Como la comparación se realiza por timestamp y no por posición dentro del archivo, este criterio se mantiene incluso si el archivo `/data.csv` fue recortado a la mitad por haber alcanzado su tamaño máximo.

De esta manera, ninguna medición tomada durante una interrupción de conectividad se pierde: queda almacenada localmente hasta que la conexión se restablece, momento en el cual se incorpora a la plataforma cloud de forma automática. Este funcionamiento es el que permite cumplir con el requisito de "offline resilience".

Resultados

A continuación se adjuntan los primeros gráficos de muestreo de los tres sensores en la página web con IP LAN:

(IP 10.52.117.152 al momento del muestreo)

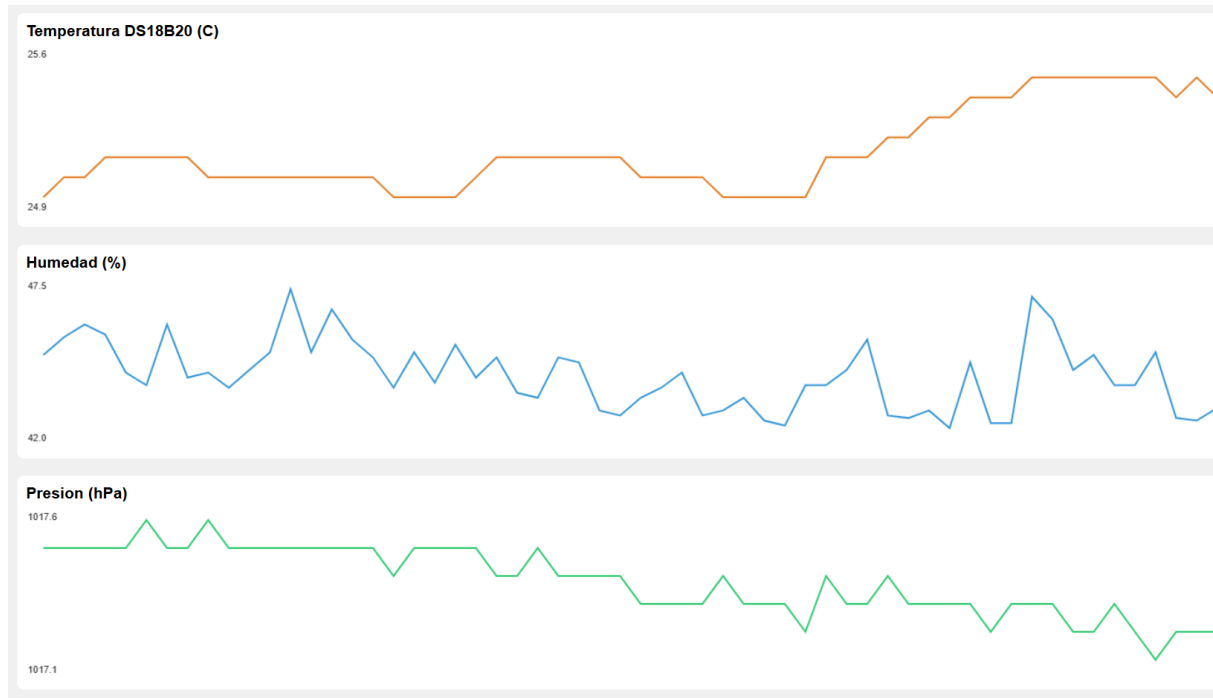


Gráfico 1: Muestreo en página web de los sensores DHT22, DS18B20, BPM280.

Gráficos de muestreo de los tres sensores en ThingSpeak:

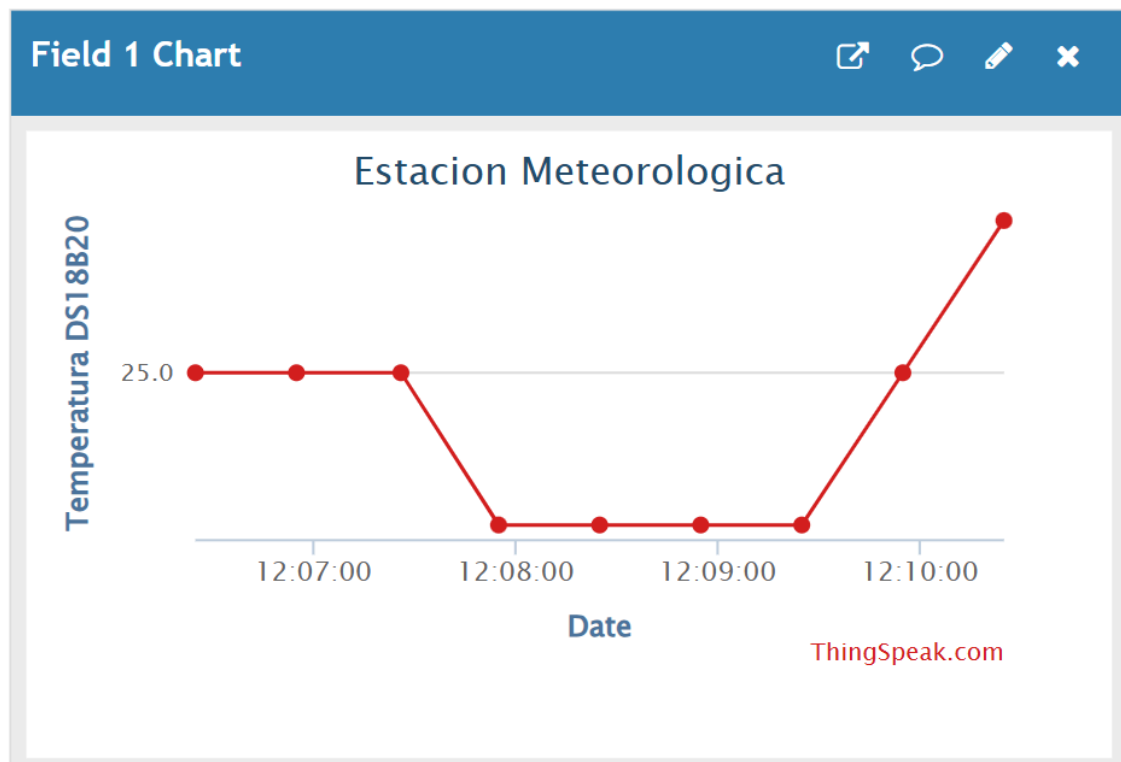


Gráfico 2A: Sensor de temperatura DS18B20

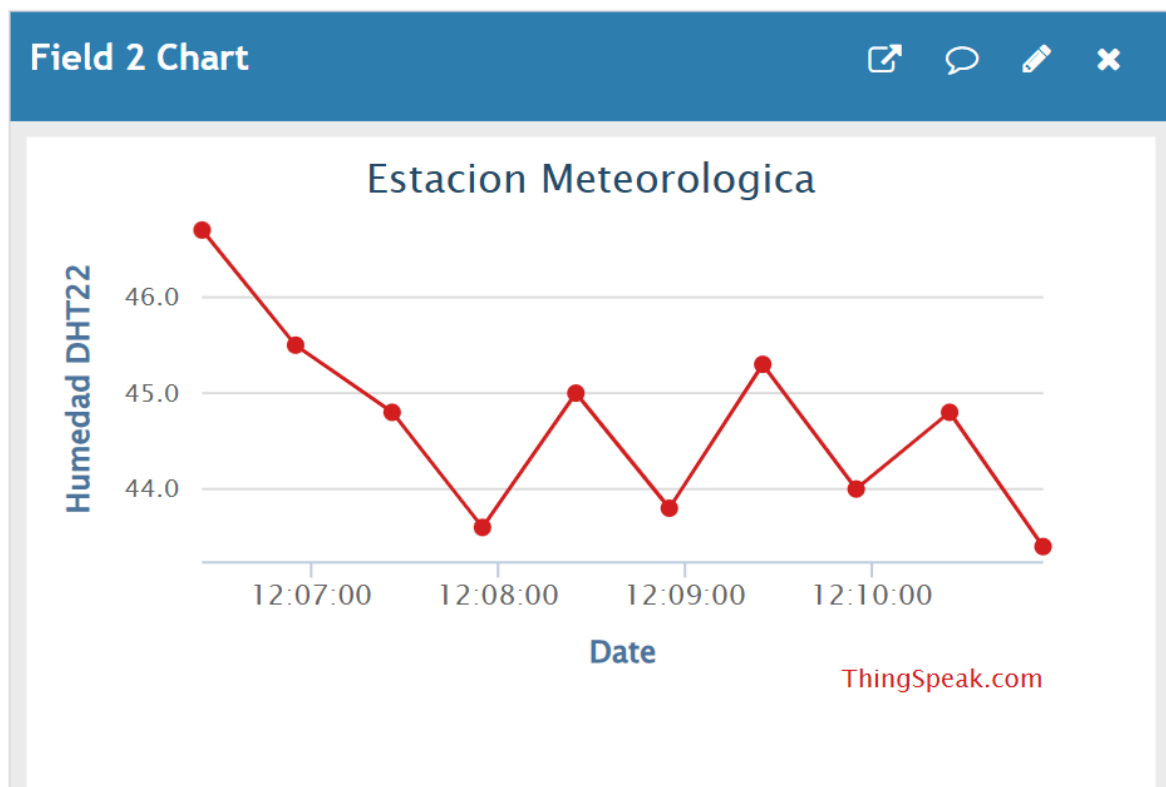


Gráfico 2B: Sensor de Humedad DHT22

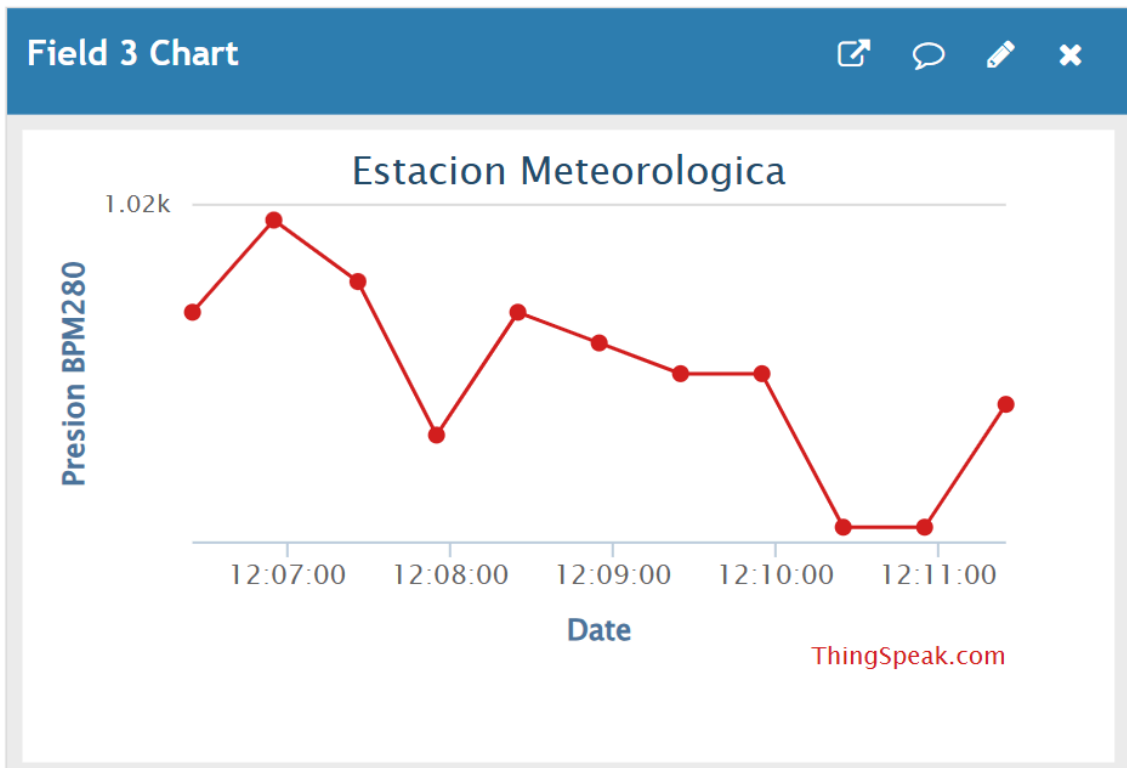


Gráfico 2C: Sensor de Presión BPM280

Tabla 1. Mediciones del entorno durante las pruebas de funcionamiento

N° prueba	Fecha y Hora	Temperatura (°C) DS18B20	Humedad (%) DHT22	Presión (hPa) BMP280
1	Sin sincronizar (arranque)	24,69	44,20	1016,88
2	26/09/2026 12:50:53	24,62	43,50	1016,91
3	26/09/2026 12:51:23	24,62	43,50	1016,88
4	26/09/2026 12:51:53	24,56	42,60	1016,91

5	26/09/2026 12:52:23	24,56	43,40	1016,85
6	26/09/2026 12:52:53	24,56	42,50	1016,85
7	26/09/2026 12:53:23	24,50	43,80	1016,89
8	26/09/2026 12:53:53	24,50	42,40	1016,88
9	26/09/2026 12:54:23	24,44	43,00	1016,85
10	26/09/2026 12:54:53	24,38	42,60	1016,81
11	26/09/2026 12:55:23	24,38	42,60	1016,79
12	26/09/2026 12:55:53	24,38	42,60	1016,78
13	26/09/2026 12:56:23	24,38	43,20	1016,83

Es importante remarcar que, tanto para la Tabla 1 como para los gráficos anteriores (1, 2A, 2B, 2C), se utilizó un intervalo de muestreo reducido (30 segundos) con el fin de agilizar las pruebas del sistema. Sin embargo, el periodo de muestreo definitivo para el proyecto es de 2 minutos. Esto se debe a que la temperatura, la humedad y la presión son variables de variación lenta, por lo que un muestreo más frecuente únicamente generaría redundancia en los datos sin aportar información adicional relevante, además de consumir espacio en memoria de manera innecesaria.

En cuanto a la primera medición en la Tabla 1, esta se realiza inmediatamente al arrancar el ESP32, antes de que pueda sincronizarse vía NTP (Network Time Protocol), protocolo el cual le permite obtener, de un servidor de internet, la información exacta de la fecha y hora actuales (en nuestro caso utilizamos pool.ntp.org). Fue una decisión consciente de diseño que se realice de esta manera, ya que de otro modo se debería esperar que transcurra un ciclo completo (es decir, el intervalo de tiempo configurado para la medición periódica) para ver los primeros datos.

Cabe aclarar que para su presentación en la Tabla 1 se convirtieron los valores de tiempo, dados en formato Timestamp Unix, a los formatos estandarizados de fecha y hora local.

Conclusiones

A partir del análisis de los resultados y el comportamiento general del prototipo, se concluye que el desarrollo de la estación meteorológica alcanzó de forma satisfactoria las metas planteadas en el proyecto. El microcontrolador ESP32 logra adquirir correctamente los datos de las tres variables ambientales mediante sus respectivos sensores y protocolos de comunicación (1-Wire con el DS18B20, bus digital con el DHT22 e I²C con el BMP280). Asimismo, se implementó de manera efectiva la doble visualización mediante el servidor web local en la red LAN y la plataforma ThingSpeak en la nube, garantizando además la "offline resilience" al almacenar las mediciones en la memoria LittleFS durante posibles interrupciones de conectividad.

Desde la perspectiva del proceso de aprendizaje, la realización de esta experiencia técnica aportó un valor significativo a nuestra formación académica. Si bien los conceptos de protocolos seriales, redes y memoria en sistemas embebidos se abordaron en las instancias teóricas de la cursada, llevarlos a la práctica en la protoboard implicó superar múltiples imprevistos cotidianos del laboratorio. Diagnosticar caídas momentáneas en las lecturas, acondicionar las señales de los pines digitales y asegurar que el código no sufriera bloqueos por falta de Wi-Fi exigieron una revisión crítica constante del diseño. Este trabajo reafirma que la práctica de taller resulta indispensable para consolidar el criterio y comprender la verdadera complejidad de un dispositivo de estas características en funcionamiento.

Mejoras a desarrollar

A partir de los resultados que obtuvimos con la estación meteorológica y tras verificar que todo funciona según lo pedido, se nos ocurren tres mejoras muy sencillas de sumar para optimizar y hacer más práctico el dispositivo.

En primer lugar, del lado del circuito, sería muy útil agregar LEDs que indiquen el estado del sistema. Como está armado ahora, la única forma de saber si el ESP32 está conectado al Wi-Fi mandando datos a ThingSpeak o si perdió la señal y empezó a guardar las mediciones en la memoria LittleFS es mirando la consola de la computadora. Conectar un LED verde para indicar cuando hay Internet y uno amarillo para el modo offline, cualquiera podría saber al instante qué está haciendo el circuito con solo mirarlo.

En segundo lugar, por el lado del programa, proponemos sumar la sincronización automática de la hora por Internet. Esto no requiere comprar ningún componente físico extra, el ESP32 simplemente obtiene la hora oficial de la red apenas se conecta y ajusta su reloj interno. De esta forma, si después se corta la conexión, el microcontrolador sigue contando el tiempo de manera interna, logrando que cada medición que se guarde en la memoria durante el corte quede registrada con la fecha y hora reales exactas en las que fue tomada.

Por último, pensamos que sería recomendable agregar una validación de datos a la lectura de presión del BMP280 (por ejemplo, descartando valores fuera de un rango físicamente posible), de modo que los tres sensores queden cubiertos por el mismo criterio de verificación.

Apéndice

El código está disponible junto con la documentación e información adicional del proyecto en una repo en GitHub.

[TL N°2 Arquitectura y Sistemas de Elaboración de Datos II](#)

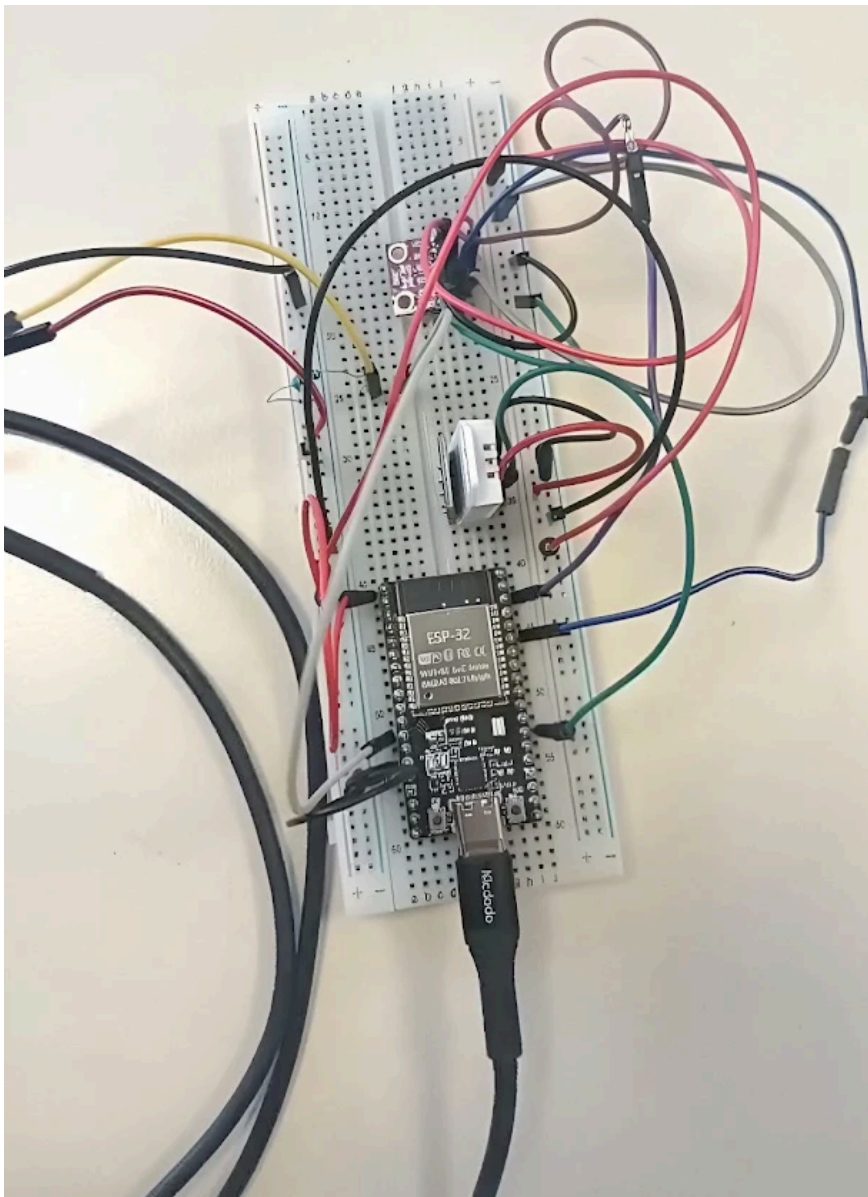


Figura 8 modelo final de la estación meteorológica

Bibliografía

[1] Arduino, «Arduino® UNO R3 Product Reference Manual», (s.f.)

<https://docs.arduino.cc/resources/datasheets/A000066-datasheet.pdf>

[2] Microcontrolador ESP32, «DeepSea Developments»

<https://www.deepseadev.com/en/blog/esp32-chip-explained-and-advantages/>

[3] Sistemas embebidos: «UNDAV — Ingeniería en Informática · Departamento de Tecnología y Administración» Docente: Ing. Matías Loiseau

[Sistemas embebidos \(Electrónica + Software\)](#)